



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ

Домашнее задание № 3

ДИСЦИПЛИНА: «Программирование на основе классов и шаблонов»

Выполнил: студент гр. ИУ5-22Б

(Подпись) (Подругин Г.Р.)
(Ф.И.О.)

Проверил:

(Гапанюк Ю.Е.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Москва, 2026 г

Тема: Разработка Windows Forms приложения для работы с реляционной базой данных с использованием Entity Framework Core и LINQ. **Предметная область:** «Справочник ресторанов и меню блюд».

1. Цель работы

Получение практических навыков проектирования и разработки десктопных приложений с графическим интерфейсом на языке C#. Освоение принципов работы с реляционными базами данных через ORM-фреймворк Entity Framework Core, реализация полного цикла CRUD-операций и формирование аналитических отчетов с помощью технологии LINQ.

2. Используемый технологический стек

- **Язык программирования:** C#
- **Платформа:** .NET
- **Пользовательский интерфейс:** Windows Forms
- **Доступ к данным:** Entity Framework Core (ORM)
- **Язык запросов:** LINQ (Language-Integrated Query)

3. Структура базы данных

В рамках задания спроектирована база данных, состоящая из двух связанных таблиц с отношением «один ко многим» (1:N):

1. **Таблица `Restaurants` (Справочник категорий):**
 - `Id` (Primary Key) — уникальный идентификатор ресторана.
 - `Name` — название ресторана.
2. **Таблица `Dishes` (Основная таблица):**
 - `Id` (Primary Key) — уникальный идентификатор блюда.
 - `Name` — название блюда.
 - `Price` — стоимость блюда.
 - `RestaurantId` (Foreign Key) — внешний ключ, связывающий блюдо с конкретным рестораном из справочника.

База данных инициализируется автоматически: при первом запуске приложения метод `EnsureCreated()` проверяет наличие файла БД и, если он отсутствует, создает структуру таблиц и заполняет их начальным набором данных.

4. Архитектура приложения и интерфейс

Приложение построено по модульному принципу. Логические блоки разделены на изолированные экранные формы, что обеспечивает интуитивно понятный UI/UX:

- **Главное меню (`Form1`):** Выполняет функцию навигатора. Содержит кнопки для перехода к управлению ресторанами, управлению блюдами и просмотру отчетов.
- **Формы отображения данных (`RestaurantsForm`, `DishesForm`):** Используют компонент `DataGridView` для вывода таблиц. Реализована автоматическая подстройка ширины столбцов (`AutoSizeColumnsMode.Fill`) и скрытие технических полей (ID).

- **Модальные окна редактирования (RestaurantEditForm, DishEditForm):** Выделенные окна для добавления и изменения записей. Выбор ресторана при создании блюда реализован через выпадающий список (ComboBox), что исключает возможность ввода несуществующего внешнего ключа.
- **Окно отчетов (ReportsForm):** Форма, содержащая три таблицы (DataGridView) для вывода результатов аналитических LINQ-запросов.

5. Программная реализация бизнес-логики

5.1. Операции с данными (CRUD)

Взаимодействие с базой данных полностью абстрагировано от SQL-кода благодаря EF Core:

- **Чтение (Read):** Данные извлекаются из базы и привязываются к UI через свойство DataSource. Для таблицы блюд используется метод `.Include(d => d.Restaurant)` (жадная загрузка) для отображения строкового названия ресторана вместо числового идентификатора.
- **Добавление/Изменение (Create, Update):** При сохранении записей реализована базовая валидация: проверка строковых полей на отсутствие пустот (`string.IsNullOrEmpty`) и проверка корректности ввода числового значения цены (`int.TryParse`). Внесенные в объекты изменения фиксируются в БД вызовом метода `SaveChanges()`.
- **Удаление (Delete) и целостность данных:** Реализована логика каскадной защиты. Перед удалением записи из таблицы `Restaurants` выполняется проверка через метод `.Any()` на наличие связанных блюд в таблице `Dishes`. При их наличии операция блокируется с выводом информационного сообщения (`MessageBox`), что предотвращает нарушение ссылочной целостности базы данных.

5.2. Формирование отчетов (LINQ-запросы)

Для выполнения требований к аналитике реализованы три отчета, которые формируются динамически при загрузке формы отчетов (`ReportsForm_Load`):

1. **Раздел 1: Полный список.** Формируется список всех блюд, соединенных с таблицей ресторанов. Применена сортировка по алфавиту названия блюда (`.OrderBy(d => d.Name)`). Данные проецируются в анонимный тип (`.Select()`) для аккуратного вывода колонок.
2. **Раздел 2: Агрегация (Количество записей).** Блюда группируются по названиям ресторанов (`.GroupBy(d => d.Restaurant.Name)`). Для каждой группы вычисляется суммарное количество позиций с помощью агрегатной функции `.Count()`.
3. **Раздел 3: Агрегация (Вычисляемое поле).** Аналогично разделу 2 производится группировка по ресторанам. Вычисляется средняя цена блюда в каждом ресторане с использованием функции `.Average(d => d.Price)`. Значение математически округляется (`Math.Round()`), а итоговый список сортируется по убыванию средней цены (`.OrderByDescending()`).

6. Вывод

В ходе выполнения домашнего задания успешно разработано десктопное CRUD-приложение, удовлетворяющее всем поставленным требованиям. На практике закреплены принципы объектно-реляционного отображения (ORM) средствами Entity Framework Core. Получены навыки построения графических интерфейсов Windows Forms с использованием механизма привязки данных (Data Binding) к визуальным компонентам. Успешно освоен синтаксис языка запросов LINQ для выборки, сортировки, группировки и агрегации данных.